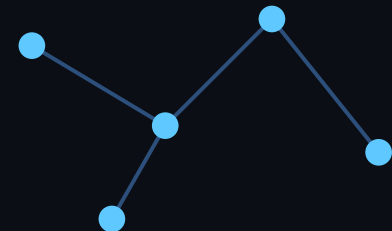


SolidJS

Frontend ramverk

Todo-app byggd med SolidJS + TypeScript



Live-demo av appen

- En enkel att-göra-lista (Todo-app)
- Lägga till en ny uppgift via ett formulär
- Checka av en uppgift - texten stryks över
- Ta bort en uppgift
- Ser hur många uppgifter som är kvar
- Meddelande visas när listan är tom
- Navigerar till "Om ramverket" via klient-routing (/om)

Vad är SolidJS?

- Skapat av Ryan Carniato, släppt 2018
- Ett reaktivt JavaScript-ramverk för att bygga användargränssnitt
- Använder JSX - ser ut ungefär som React
- Ingen Virtual DOM - uppdaterar riktig DOM direkt och exakt
- Byggs ofta med Vite som utvecklingsserver / byggverktyg

SolidJS vs React - kort sagt

- React: komponentfunktionen körs OM ("re-render") vid varje ändring, med hjälp av en Virtual DOM som jämför gammalt/nytt
- SolidJS: komponentfunktionen körs bara EN gång - reaktiva "signals" uppdaterar sedan bara exakt den DOM-nod som ändrats
- Resultat: SolidJS slipper hela re-render-steget, vilket ofta gör det snabbare
- Syntaxen (JSX, komponenter, props) känns väldigt lik React

SolidJS vs React - jämförelse

Aspekt	SolidJS	React
Rendering	Fine-grained reactivity, ingen Virtual DOM	Virtual DOM med diffing
Vid state-ändring	Bara berörd DOM-nod uppdateras	Hela komponenten körs om
State	<code>createSignal()</code>	<code>useState()</code>
Community	Litet, växande	Enormt, industristandard
Routing	<code>@solidjs/router</code>	<code>react-router</code>

Databindning

Kopplar ett värde i state till ett HTML-element, t.ex. ett textfält

src/App.tsx

```
<input  
  value={text()}  
  onInput={(e) => setText(e.currentTarget.value)}  
>
```

Villkorlig rendering

Visar olika innehåll beroende på ett villkor

src/App.tsx

```
<Show when={todos().length > 0} fallback={<p>Inga uppgifter än.</p>}>  
  ...  
</Show>
```

Loop rendering

Ritar upp en lista av komponenter - en per objekt i en array

src/App.tsx

```
<For each={todos()}>  
  {(todo) => <TodoItem ... />}  
</For>
```


Klass- och stilbindning

Sätter en CSS-klass villkorligt - genomstruken text på avklarade uppgifter

src/components/ToDoItem.tsx

```
<li classList={{ done: props.done }}>  
  ...  
</li>
```

HUR DET FUNGERAR

Tillståndshantering

Data som kan ändras och som appen "kommer ihåg" och reagerar på

src/App.tsx

```
const [todos, setTodos] = createSignal<Todo[]>([])  
const [text, setText] = createSignal('')
```

HUR DET FUNGERAR

Eventhantering

Kör kod när användaren gör något - klickar eller skickar ett formulär

src/App.tsx / TodoItem.tsx

```
<form onSubmit={addTodo}>  
<button onClick={props.onRemove}>x</button>
```

HUR DET FUNGERAR

Komponenter

Återanvändbar byggsten som tar emot props och renderar UI

src/components/ToDoItem.tsx

```
function ToDoItem(props: ToDoItemProps) {  
  return <li>...</li>  
}
```

Klient-routing

Låter appen visa olika sidor/komponenter beroende på URL:en, utan att ladda om sidan

src/index.tsx

```
<Router>  
  <Route path="/" component={App} />  
  <Route path="/om" component={About} />  
</Router>
```

Community, dokumentation & framtid

- Community: mindre än Reacts, men aktivt och växande - Discord-community
- Dokumentation: tydlig och beginner-vänlig på solidjs.com
- Skapare Ryan Carniato jobbar numer även med Solid Start (fullstack-ramverk ovanpå SolidJS)
- Används av bl.a. mindre startups och prestandakänsliga projekt
- Växer stadigt i popularitet tack vare sin prestanda och enkla mentala modell

Sammanfattning

- SolidJS liknar React i syntax men fungerar helt annorlunda under huven
- Signals + ingen Virtual DOM = mindre omkörning, ofta bättre prestanda
- Byggde en Todo-app som täcker alla 7 kärnkoncept + klient-routing (VG-nivå)
- Lärde mig att tänka reaktivt istället för i re-renders

Vi hinner tyvärr inte med frågor!  

Tack för mig!